MODULE 14

DTOs, Validation and API Contracts

Design effective Data Transfer Objects, implement robust validation, and define clear API contracts.

TEKsystems Global Services

WORKFORCE DEVELOPMENT

MODULE 14 OVERVIEW

Designing strong, predictable, and reliable APIs — DTOs, validation, and API contracts.

- Learn how to design effective Data Transfer Objects, implement robust validation, and define clear API contracts to build secure, maintainable, consumer-friendly services.

DURATION & LAB

1 Hour Theory

DTOs, Bean Validation, API contracts.

2 Hours Lab

DTOs and Validation.

Scenario

Build validated Customer DTOs for Northstar CRM.

DTO DESIGN
Purpose-driven, simple, immutable, version-friendly

VALIDATION
Jakarta Validation (formerly JSR 380) at the API boundary

CONTRACTS
Consistent, versioned agreements with consumers

KEY TAKEAWAY Well-designed DTOs, strong validation, and clear API contracts are the foundation of reliable, secure APIs.

WHY DTOS MATTER

DTOs act as a contract between your API and its consumers — build APIs that are secure, maintainable, and consumer-friendly.

WITHOUT DTOS

- Over-exposure of sensitive fields (passwords, audit data)
- Tight coupling — consumers depend on internal models
- Poor performance — unnecessary/heavy data
- Difficult to evolve without breaking clients

VS

WITH DTOS

- Data security — expose only what's needed
- Loose coupling — a stable contract
- Better performance — smaller payloads
- Easier evolution and versioning

COMMON USE CASES FOR DTOS

Mobile Apps

Lightweight data tailored for constrained clients.

Public APIs

Expose safe, versioned data to external partners.

Multiple Views

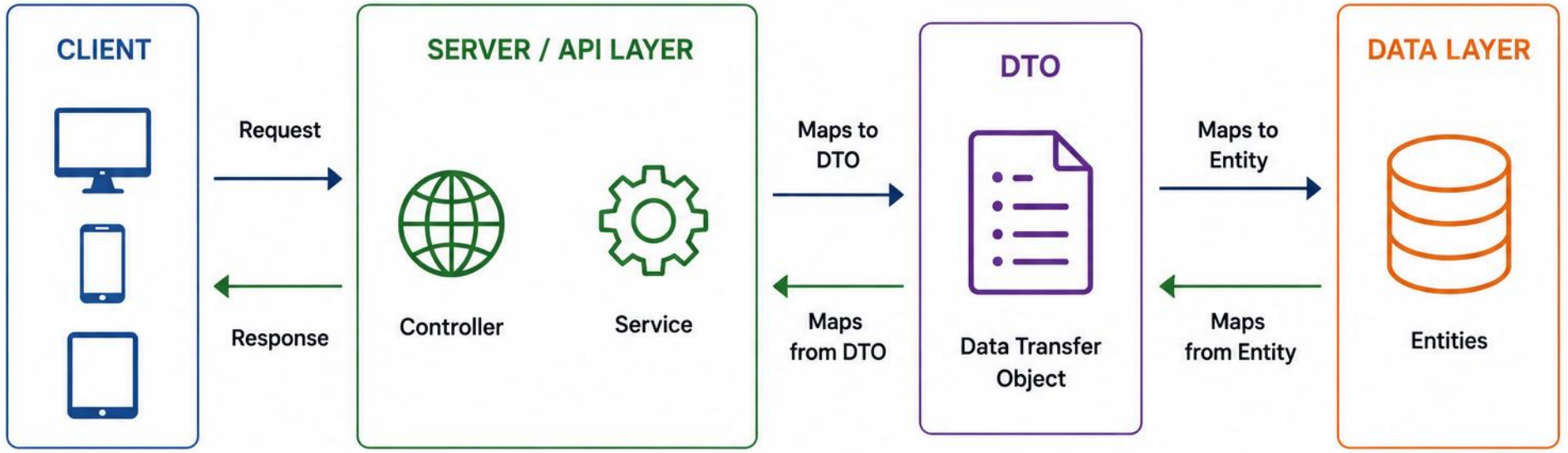
Different DTOs for list, detail, or report views.

Microservices

Transfer only relevant data between services.

KEY TAKEAWAY Use DTOs to define clear contracts, protect your domain, and deliver the right data to the right consumer — every time.

DTOs (Data Transfer Objects)



WHY DTOs MATTER

Security
Hide internal data and sensitive fields

Performance
Transfer only the required data

Loose Coupling
Decouple API from internal models

Flexibility
Evolve internal models without breaking APIs

Maintainability
Simpler, cleaner and easier to maintain

ENTITY VS DTO

Use Entities to model your business and store data. Use DTOs to communicate data safely and efficiently.

ASPECT	ENTITY	DTO
Purpose	Model the domain & persist data	Transfer data across boundaries
Scope	Internal (application)	External (API / clients)
Contains	Entities represent persisted domain state and may include fields or relationships that should not be exposed through an API.	Only required fields
Relationships	Yes (to other entities)	No, or simplified
Change Impact	High (affects many areas)	DTOs can evolve independently from entities, but public contract changes must still be managed for compatibility.

ENTITY VS DTO — EXAMPLES & BEST PRACTICES

Concrete Entity/DTO name pairs and guidance for choosing dedicated DTOs over exposing persistence entities directly.

EXAMPLES

- Entities: UserEntity, OrderEntity, ProductEntity → DTOs: UserResponseDto, CreateUserRequestDto, UserSummaryDto.

BEST PRACTICES

- For public, partner-facing, or long-lived APIs, use dedicated DTOs rather than exposing persistence entities directly. Convert between entities and DTOs through explicit mapping logic.

KEY TAKEAWAY Entities model your world. DTOs shape your communication — use the right tool in the right place.

Entity vs DTO

ENTITY		DTO
Represents domain data and business state	Purpose	Carries data between layers or over the network
Used within the domain (Entity Layer)	Scope	Used across layers or external boundaries
May contain business logic, rules, and behaviors	Behavior	No business logic, only data
May have relationships with other entities	Dependencies	No references to other domain objects
Persisted in the database (mapped by ORM)	Persistence	Not persisted, used for data transfer only
Rich structure with many fields and relationships	Structure	Flat or simple structure with only required fields
Used by Service/Repository layers for domain operations	Use Case	Used by Controller, API, or client layers
Changes can impact domain logic and persistence	Change Impact	Changes are isolated, safe for external contracts

TRY IT YOURSELF — EXERCISE 1: ENTITY VS DTO

Reinforces: the Entity vs DTO purpose, scope, and contents you just saw.

STEP	WHAT TO DO
Goal	Explain why Northstar HTTP/SOAP payloads should not be persistence entities
File	notes/lab14-entity-vs-dto.md (in examples/module-14-exercises/)
Leak risks	Two leaks: internal flags, lazy relations, or audit columns in responses
Amina fields	customerId, fullName, status — no persistence annotations
Boundary	Pre-lab only — no Spring @Valid controllers yet
Reference	exercises/exercise-01-entity-vs-dto.md

```
// CustomerEntity (persistence) vs CustomerDto (API) for Amina Khan
class CustomerEntity { Long id; String fullName; CustomerStatus status; }
  boolean internalRiskFlag; Instant auditCreatedAt;  // never leaves this class
record CustomerDto(String customerId, String fullName, String status) {}
// no persistence annotations on the DTO
```

TRY IT NOW Complete Exercise 1 before continuing — see exercises/exercise-01-entity-vs-dto.md.

REQUEST DTO DESIGN

A Request DTO captures the data sent by the client for a specific operation — validates input and protects your domain model.

PRINCIPLES

- Client-controlled fields only — the client supplies just what it's allowed to set.
- Validation-ready — annotate fields with Jakarta Validation constraints.
- Use separate create, replace, and patch contracts when their required fields or semantics differ.
- Over-posting protection — never bind server-controlled or internal fields from client input.

EXAMPLE REQUEST DTO

```
public record CreateCustomerRequest(
    @NotBlank String fullName,
    @NotBlank @Email String email
) {}
// Records suit immutable request/response DTOs;
// check framework & serialization support.
```

REQUEST DTO DESIGN — WHAT TO EXCLUDE & PATTERNS

Fields client input must never set, and the standard shapes of create, replace, and patch requests.

WHAT TO EXCLUDE

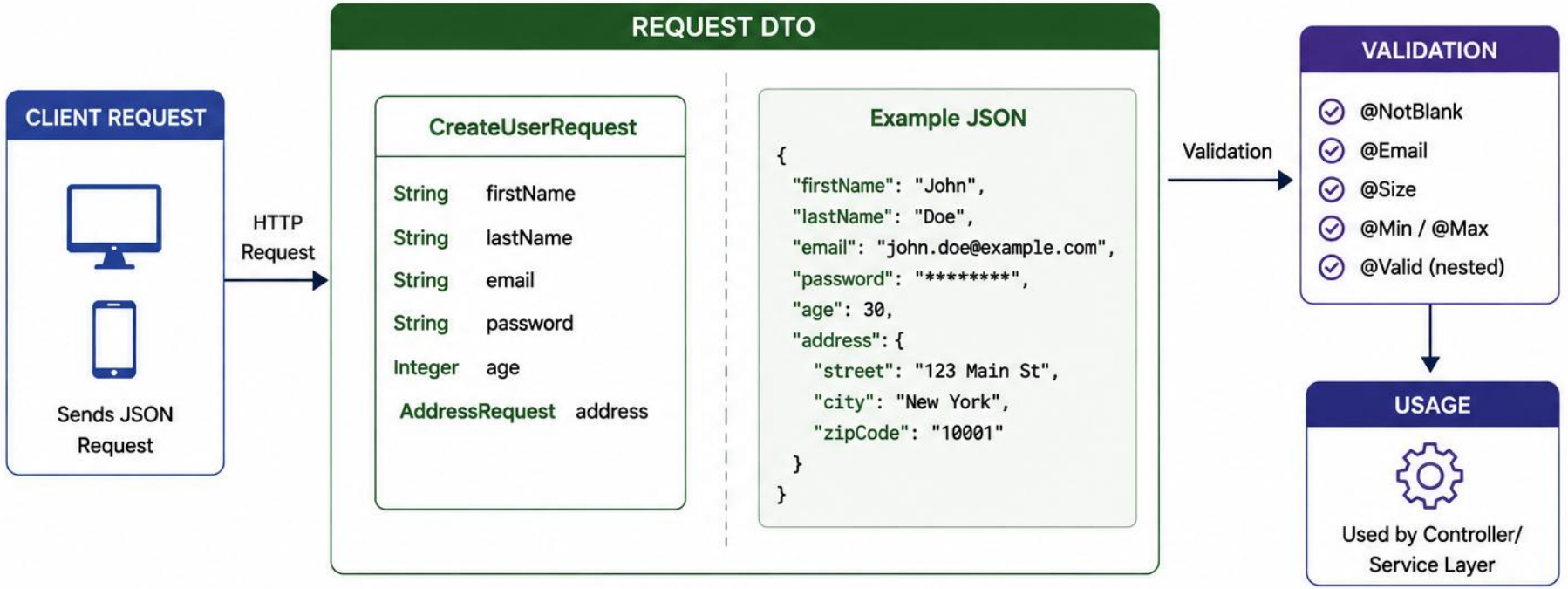
- Server-generated fields (id, createdAt, status); sensitive fields (internal flags, audit info) that the client should never set.

COMMON REQUEST PATTERNS

- Create Request (POST) — full resource creation. Replace Request (PUT) — full resource replacement. Patch Request (PATCH) — partial modification; null may mean absent, explicit clearing, or invalid depending on design. Use JSON Merge Patch, JSON Patch, explicit optional wrapper types, or field-presence tracking for patch — don't reuse a normal nullable-field DTO unless absence-vs-explicit-null is clearly handled.

KEY TAKEAWAY Design Request DTOs around client-controlled fields, validation, and clear create/replace/patch semantics.

Request DTO Design



BEST PRACTICES

1. SECURITY
Include only required fields. Never expose internal data.

2. VALIDATION
Validate all inputs using Bean Validation annotations.

3. LOOSE COUPLING
Keep DTOs separate from entities to avoid coupling and protect internal model.

4. SIMPLICITY
Keep DTOs focused, small and easy to understand.

5. EXTENSIBILITY
Design for future changes using optional fields and versioning.

RESPONSE DTO DESIGN

A Response DTO defines the data returned after a request is processed — consistent, secure, and efficient.

PRINCIPLES

- Server-controlled fields only — computed or persisted values the client cannot set.
- Consistent representation shape — same structure for the same resource type.
- Sensitive-data filtering — never expose passwords, internal flags, or stack traces.
- Pagination and links — include paging metadata and navigation links for collections.

EXAMPLE SUCCESS RESPONSE

```
// Direct resource response
{ "id": 1, "name": "Jane" }

// Standard envelope (when required)
{ "data": { ... }, "meta": { ... } }
```

COMMON PATTERNS

- Standard response, paged response (with pagination), summary/count response, action result response.

RESPONSE DATA ELEMENTS

- Also include metadata (pagination, timestamps, correlationId) and HATEOAS links when applicable. Avoid a redundant textual status field when the HTTP status code already communicates success. Additive fields are generally backward-compatible when consumers tolerate unknown properties. Confirm client behavior and published schema rules.

KEY TAKEAWAY A great API doesn't just process requests — it communicates effectively. Return the right data, in the right format.

Response DTO Design

SERVER / SERVICE

Service Logic

Data / Entity

Maps to DTO

RESPONSE DTO

Example Response DTO Structure

```
class UserResponse {
    Long id;
    String firstName;
    String lastName;
    String email;
    Integer age;
    AddressResponse address;
    List<String> roles;
}
class AddressResponse {
    String street;
    String city;
    String state;
    String zipCode;
}
```

Example JSON Response

```
{
  "id": 1001,
  "firstName": "John",
  "lastName": "Doe",
  "email": "john.doe@example.com",
  "age": 30,
  "address": {
    "street": "123 Main St",
    "city": "New York",
    "state": "NY",
    "zipCode": "10001"
  },
  "roles": ["USER", "ADMIN"]
}
```

Returned to Client

CLIENT

Receives JSON Response

DESIGN PRINCIPLES

Expose Only Necessary Data
Return only the fields required by the client.

Hide Internal Details
Do not expose sensitive or internal data.

Loose Coupling
DTOs decouple API contracts from domain models.

Consistent Structure
Use consistent naming and format across APIs.

BEST PRACTICES

- Use meaningful and consistent field names.
- Support pagination for large collections.
- Include metadata when useful.
- Use nested DTOs for related data.
- Version your API responses when needed.

COMMON RESPONSE ENVELOPE

```
{
  "success": true,                          // Status of the request
  "message": "Request successful",          // Message for the client
  "data": {
     ...  // Response DTO                   // Response DTO payload
  },
  "metadata": {
    "timestamp": "2025-05-21T10:15:30Z",    // Additional metadata
    "requestId": "a1b2c3d4-e5f6-7890"
  }
}
```

MAPPING STRATEGIES

Convert Entities to DTOs (and back) using the right approach for your complexity, team, and performance needs.

STRATEGY	CHARACTERISTICS	SUITABLE FOR
`Manual Mapping`	Explicit, no extra framework, more boilerplate	Small mappings or complex custom logic
`ModelMapper`	Convention-based, runtime mapping	Prototypes or simple mappings
`MapStruct`	Compile-time generated, type-safe	Larger production codebases
`Legacy reflection tools`	Flexible but harder to reason about	Existing legacy systems

MAPPING STRATEGIES — BEST PRACTICES & FLOW

How to keep mapping logic maintainable, and the standard Entity-to-DTO-to-consumer flow.

BEST PRACTICES

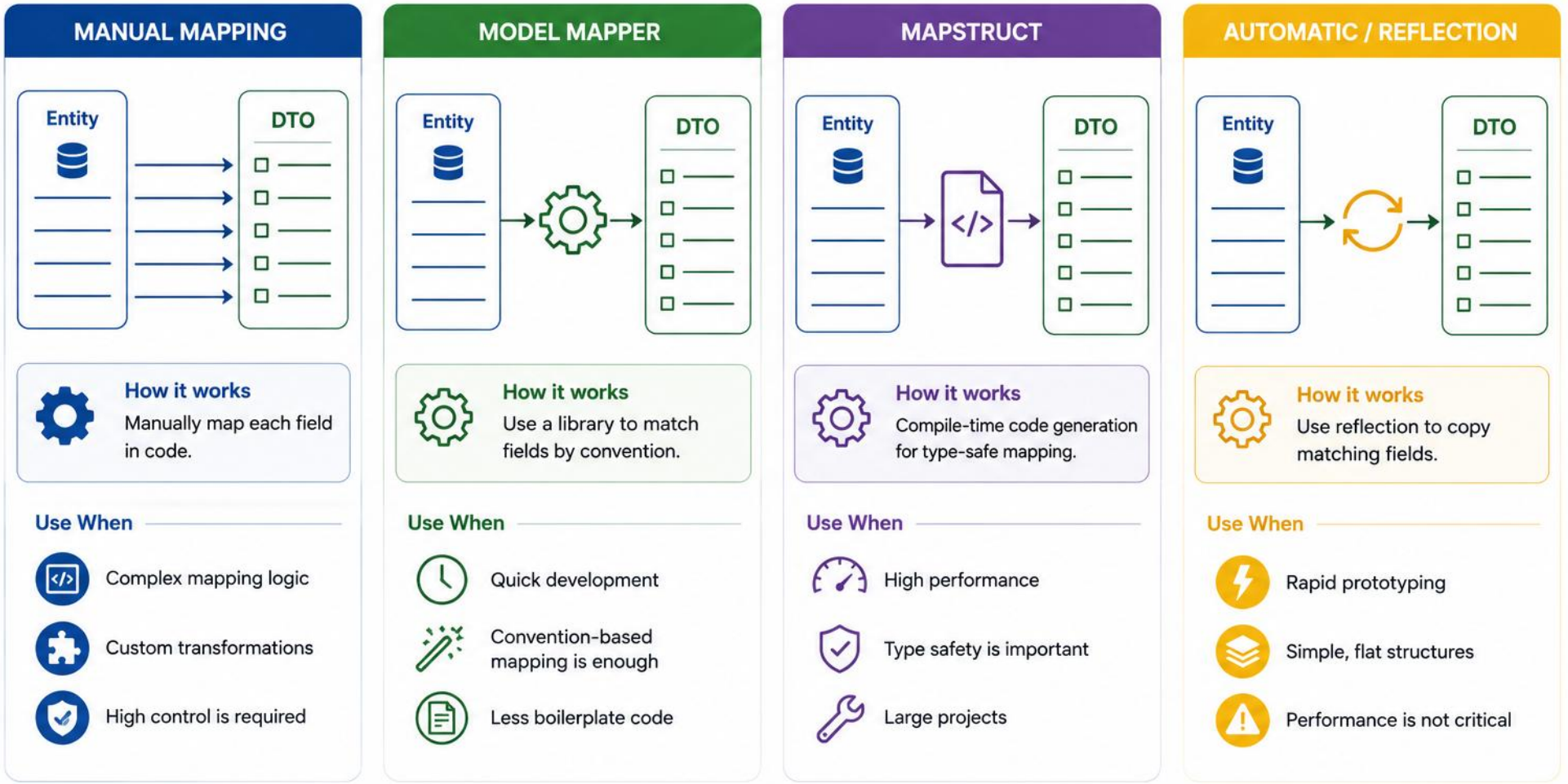
- Keep mapping logic in a dedicated layer; validate before mapping to Entities.
- MapStruct is a strong choice for repetitive, type-safe mappings. Use manual mapping when transformation rules are small, security-sensitive, or highly domain-specific.
- Handle nulls and collections carefully; write unit tests for custom mappings.

MAPPING FLOW

- Entity → Mapping Layer → DTO → API Consumer. Treat request-to-domain and domain-to-response mapping as separate operations with separate security and business rules — clients cannot set IDs, audit fields are server-generated, existing entity state is preserved, patch semantics need field-presence handling, and relationships may be resolved through services.

KEY TAKEAWAY Choose the mapping strategy that balances productivity, maintainability, performance, and team expertise.

Mapping Strategies



TRY IT YOURSELF — EXERCISE 2: MAPPER NO-LEAK RULE

Reinforces: the mapping strategies and Entity→DTO flow you just saw.

STEP	WHAT TO DO
Goal	Sketch toDto/toEntity rules that keep internals out of API responses
File	notes/lab14-mapper-rules.md (in examples/module-14-exercises/)
toDto (CUS-1001)	Map only id, fullName, status — nothing else
Forbidden	Password hashes, internal risk scores, raw SQL ids if different from public id
Activate DTO	Carries customerId only (+ correlation header outside the body)
Reference	exercises/exercise-02-mapper-no-leak.md

```
// CustomerMapper -- no-leak rule for CUS-1001
CustomerDto toDto(CustomerEntity e) {
  return new CustomerDto(e.getCustomerId(), e.getFullName(), e.getStatus().name());
}
// Never map passwordHash/riskScore -- DTOs before deep rules (Lab 15 owns transitions)
```

TRY IT NOW Complete Exercise 2 before continuing — see exercises/exercise-02-mapper-no-leak.md.

JAKARTA VALIDATION OVERVIEW

Validate early. Fail fast. Jakarta Validation, formerly Bean Validation and standardized in earlier versions through JSR 380, validates Java objects using annotations.

KEY TAKEAWAY Jakarta Validation is declarative, centralized, and consistent — apply it at the API boundary before business logic runs.

JAKARTA VALIDATION OVERVIEW — CONSTRAINTS & TARGETS

The built-in constraint annotations, and where structural validation applies.

COMMON BUILT-IN CONSTRAINTS

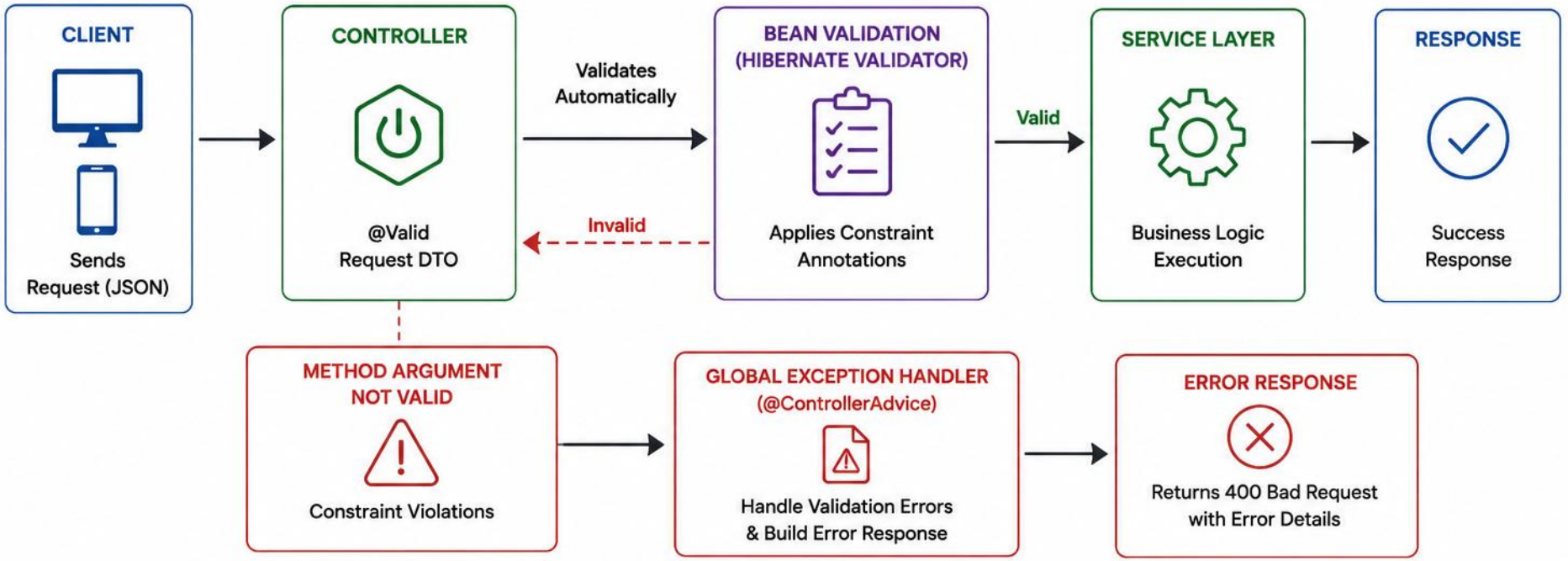
ANNOTATION	DESCRIPTION
`@NotNull / @NotBlank / @NotEmpty`	Value/String/Collection must not be null (or blank/empty)
`@Size(min, max)`	Size or length must be within range
`@Email / @Pattern(regexp)`	Must be a valid email / match the given regex

VALIDATION TARGETS & TYPES

- Validates fields, method parameters, return values, nested objects (@Valid), and class-level cross-field rules (@AssertTrue).
- Use Jakarta Validation for structural and local cross-field constraints. Enforce business rules in the domain or service layer where business context is available.

KEY TAKEAWAY Jakarta Validation is declarative, centralized, and consistent — apply it at the API boundary before business logic runs.

Bean Validation



COMMON CONSTRAINT ANNOTATIONS	
@NotNull	must not be null
@NotBlank	must not be null or blank (String)
@NotEmpty	must not be null or empty (Collection, Map, Array, String)
@Size(min, max)	size must be between min and max
@Min(value)	must be greater than or equal to value
@Max(value)	must be less than or equal to value
@Email	must be a well-formed email address
@Pattern(regepp)	must match the given regular expression

VALIDATION TARGETS

- Request DTO Fields
- Method Parameters
- Method Return Values
- Nested Objects (@Valid)

KEY BENEFITS

- Ensures data integrity
- Automatic validation
- Reduces boilerplate code
- Centralized error handling
- Improves API reliability

PRESENCE CONSTRAINTS

Ensure required data is present — @NotNull, @NotEmpty, and @NotBlank guard against missing, empty, or blank values.

EXAMPLE

```
public record CreateUserRequest(
    @NotBlank String name
) {}

@NotNull
@Valid
private AddressRequest address;
```

WHERE CAN YOU USE IT?

- Entity/DTO fields, constructor parameters; controller or service parameters (needs framework support).
- Invalid: null, "" (empty), " " (whitespace only).
- Valid: any string with at least one non-whitespace character, e.g. "John Doe".
- @NotNull requires presence; @Valid validates the nested object — they are independent concerns.

PRESENCE CONSTRAINTS — @NOTNULL VS @NOTBLANK VS @NOTEMPTY

A side-by-side comparison of the three presence-checking annotations and what each one allows.

@NOTNULL VS @NOTBLANK VS @NOTEMPTY

ANNOTATION	APPLIES TO	ALLOWS EMPTY?	ALLOWS WHITESPACE?
`@NotNull`	Any type	Yes	Yes
`@NotEmpty`	CharSequence, collections, maps, arrays	No	Yes
`@NotBlank`	String	No	No

- @NotNull requires presence; @Valid validates the nested object — they are independent concerns (see the combined example above).

KEY TAKEAWAY @NotNull, @NotEmpty, and @NotBlank ensure required data is present — combine with @Valid for nested objects.

@NotNull Annotation

@NotNull Annotation

@NotNull

Marks a field, parameter or return value as required

Validation (Runtime)

Validation checks if the value is null

Result

If null → Validation fails and an error is thrown

Example Usage

```
public class UserDTO {
    @NotNull
    private String username;

    @NotNull
    private String email;
}
```

Effect

- Prevents null values
- Ensures required data
- Improves data integrity

@NotBlank Annotation

@NotBlank Annotation

Marks a field, parameter or return value as required

Validation ensures the value is not null, not empty, and contains at least one non-whitespace character.

Validation (Runtime)

Validation checks if the value is:

- Not null
- Not empty ("")
- Not only whitespace (" .. ")

Result

If validation fails

ConstraintViolationException is thrown and the request is rejected.

Example Usage

```
public class UserDTO {
  @NotBlank(message = "Username is required")
  private String username;

  @NotBlank(message = "Email is required")
  private String email;

}
```

Behavior Examples

Input Value	Valid?	Reason
"John"	✓	Valid
" John "	✓	Valid (contains non-whitespace)
"" (empty)	✗	Invalid (empty)
" .. " (spaces)	✗	Invalid (only whitespace)
null	✗	Invalid (null)

Benefits

- Ensures meaningful text input
- Prevents empty or whitespace values
- Improves data quality
- Reduces manual validation code

FORMAT AND SIZE CONSTRAINTS

Validate format and size — @Email, @Pattern, @Size, and @Min/@Max/@Positive keep data well-formed and within range.

EXAMPLE

```
@NotBlank
@Size(min = 3, max = 50)
private String name;


@Email
@NotBlank
private String email;
```

VALID VS INVALID

- Valid: john.doe@example.com, jane_doe123@domain.co.uk
- Invalid: john.doe@, john.doe.com, john..doe@example.com

BEST PRACTICES

- @Size applies to String length, Collection/Array element counts, and Map entries; @Pattern(regexp) matches a custom regular expression; @Min, @Max, and @Positive validate numeric ranges.
- Most format and size constraints treat null separately. Add a presence constraint when the value is required.
- @Email performs a basic syntax check. Verification links or other workflows are required to confirm ownership or deliverability.
- Combine @Email with a presence constraint when the field is required; always validate at the API boundary using @Valid.

KEY TAKEAWAY **Format and size constraints keep data well-formed and within range — combine with a presence constraint when the value is required.**

@Email Annotation

@Email Annotation

Marks a field, parameter, or return value to be validated as a valid email address.

Validation (Runtime)

Validation checks if the value is a valid email address format.

Valid Example

user@example.com

Result

If validation fails, a ConstraintViolationException is thrown and the request is rejected.

Invalid Example

user@, userexample.com

Validation Rule

- Must not be null
- Must follow standard email format
- Checks local part, domain and TLD

Example Usage

```
import jakarta.validation.constraints.Email;

public class UserDTO {
   @Email(message = "Invalid email address")
   private String email;
}
```

Behavior Examples

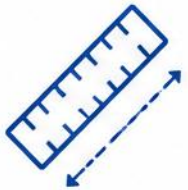
Input Value	Valid?	Reason
user@example.com	✓	Valid email format
user.name@domain.co.in	✓	Valid email format
user@domain	✗	Missing top-level domain
user@.com	✗	Invalid domain
@example.com	✗	Missing local part
userexample.com	✗	Missing @ symbol

Benefits

- Ensures email format is correct
- Prevents invalid data from entering system
- Reduces manual validation code
- Improves API reliability and data quality

@Size Annotation

@Size Annotation



Validates the size of Strings, Collections, Arrays, or Maps.

Validation (Runtime)

Checks if the size/length is within the specified min and max limits.

`@Size(min = X, max = Y)`
`min ≤ size ≤ max`

Result

If validation fails, a ConstraintViolationException is thrown and the request is rejected.

Size must be between min and max (inclusive).

Applies To

- String
- Collection (List, Set, etc.)
- Array
- Map

Example Usage

```
import jakarta.validation.constraints.Size;

public class UserDTO {
   @Size(min = 3, max = 20,
         message = "Name must be between
         3 and 20 characters")
   private String name;

   @Size(min = 1, max = 5)
   private List<String> roles;
}
```

Behavior Examples

Input Value	@Size(min = 2, max = 5)	Result
"Hi" (length = 2)	✓	Valid
"Hello" (length = 5)	✓	Valid
"H" (length = 1)	✗	Invalid (too short)
"Welcome" (length = 7)	✗	Invalid (too long)
[A, B, C] (size = 3)	✓	Valid
[] (size = 0)	✗	Invalid (too short)

Key Benefits

- Ensures data size/length is within allowed limits
- Prevents invalid or excessive data
- Reduces manual validation code
- Improves API reliability and data quality

TRY IT YOURSELF — EXERCISE 3: ANNOTATE THE PAPER DTO

Reinforces: the presence, format, and size constraints you just saw.

STEP	WHAT TO DO
Goal	Mark required/optional constraints on a paper CreateCustomerRequest
File	notes/lab14-paper-dto.md (in examples/module-14-exercises/)
fullName	required, non-blank → @NotBlank
status	optional on create; defaults to PROSPECT (Ravi starts PROSPECT)
customerId	server-assigned or pattern CUS-#### → @Pattern
Reference	exercises/exercise-03-annotate-paper-dto.md

```
// Paper DTO -- documentation only, no Spring @Valid wired yet
record CreateCustomerRequest(
    @NotBlank String fullName, String status,  // status optional, defaults PROSPECT
    @Pattern(regexp = "CUS-####") String customerId  // server-assigned
) {}  // correlation lab-request-001 stays in headers/logs, not a field
```

TRY IT NOW Complete Exercise 3 before continuing — see exercises/exercise-03-annotate-paper-dto.md.

VALIDATION ERROR HANDLING

When validation fails, return a standard, machine-readable error response with stable codes and a traceable correlation ID.

EXAMPLE ERROR RESPONSE

```
{
  "type": "https://api.northstar.example/problems/validation",
  "title": "Validation failed",
  "status": 400,
  "detail": "One or more fields are invalid.",
  "instance": "/api/v1/customers",
  "errors": [
    { "field": "email", "message": "must be a well-formed email address" }
  ],
  "correlationId": "lab-request-001"
}
```

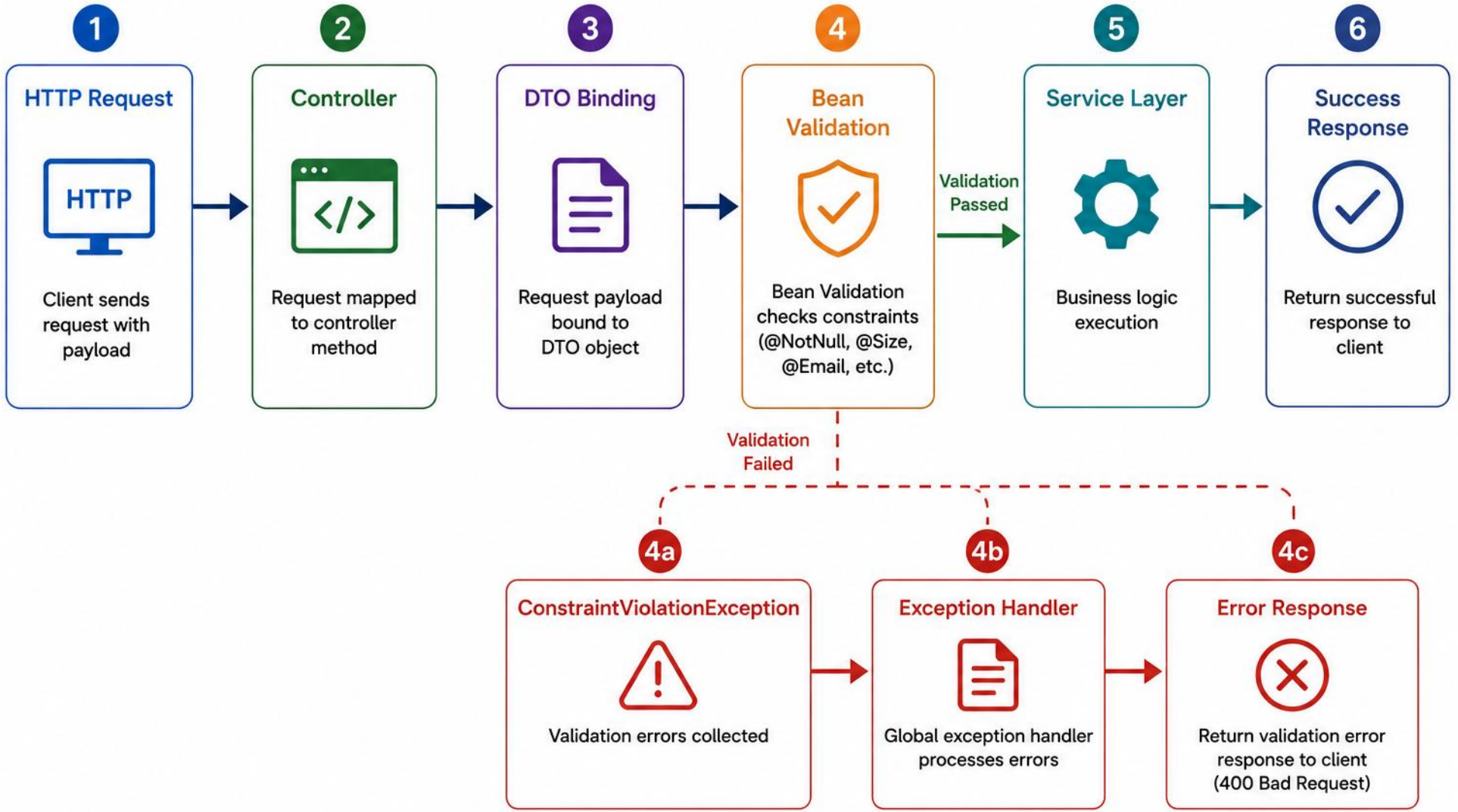
- "errors" and "correlationId" are extensions — non-standard fields added on top of the RFC 9457 base.

STABLE ERROR CODES & CORRELATION ID

- Define a stable external error code, consumer-safe message, field path, correlation ID, and optional localized text — never expose raw internal validation messages, e.g. { "code": "CUSTOMER_EMAIL_INVALID", "field": "email", "message": "Enter a valid email address." }
- Correlation ID comes from an incoming request header or is generated at the boundary; include it in logs, error responses, and service calls — never use it as a security token.

KEY TAKEAWAY Return RFC 9457 problem details with stable error codes and a correlation ID so every rejection is traceable and safe to show clients.

Validation Workflow



TRY IT YOURSELF — EXERCISE 4: INVALID CASES CATALOG

Reinforces: the validation error handling and RFC 9457 response you just saw.

STEP	WHAT TO DO
Goal	Catalog invalid requests you will assert later in Lab 14
File	notes/lab14-invalid-cases.md (in examples/module-14-exercises/)
Create invalids	Blank name; status = "ACTVE" typo; oversized name
Activate invalids	Activate missing id; activate CUS-9999 unknown
Valid control	Create Ravi-shaped PROSPECT with a non-blank name
Reference	exercises/exercise-04-invalid-cases.md

```
// Invalid create payloads to assert in Lab 14
{ "fullName": "", "status": "PROSPECT" }  // blank name
{ "fullName": "Amina Khan", "status": "ACTVE" }  // status typo
{ "fullName": "A...A" }  // 300 chars, oversized name
// Valid control: { "fullName": "Ravi Shah", "status": "PROSPECT" }
```

TRY IT NOW Complete Exercise 4 before continuing — see exercises/exercise-04-invalid-cases.md.

HTTP API CONTRACT DESIGN

An API contract is a formal agreement between provider and consumer. This section focuses on REST-style HTTP contracts.

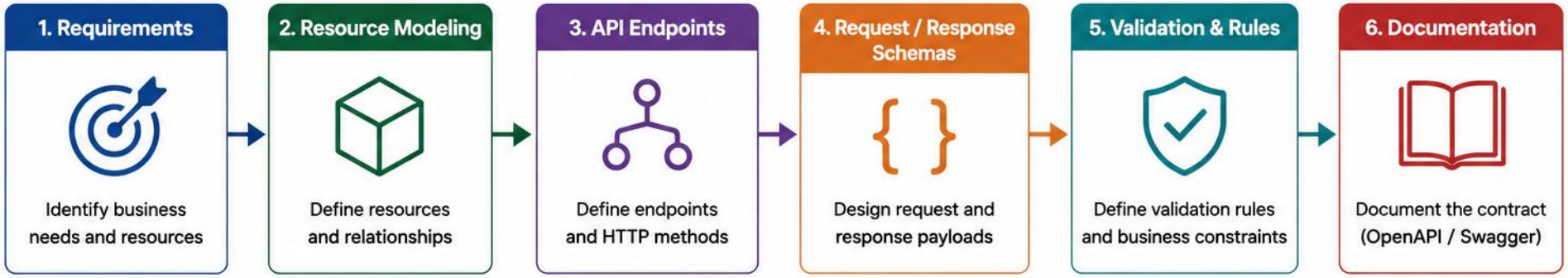
COMPONENT	DEFINES	EXAMPLE
`Endpoints`	HTTP method, path, query parameters	POST /api/v1/users
`Request Contract`	Request DTO structure, validation rules	name, email, age fields
`Response Contract`	Response DTO structure, pagination	201 Created + UserResponseDto
`Error Contract`	Standard error structure, codes	400 Bad Request + field errors
`Versioning`	URI or header versioning strategy	/api/v1/users

BEST PRACTICES

- Keep contracts simple and consistent; use nouns for resources and standard HTTP methods; document with OpenAPI/Swagger.
- Specify content types, authentication/authorization expectations, and idempotency for unsafe methods.
- Define pagination, rate limits, field semantics, and nullability for every resource.
- Provide request/response examples, a deprecation policy, and clear compatibility guarantees.

KEY TAKEAWAY A clear API contract ensures consistency, reduces integration issues, and speeds up development.

API Contract Design



API Contract Components

- **Base URL** — API base endpoint
- **Endpoints** — Resource paths and operations
- **HTTP Methods** — GET, POST, PUT, PATCH, DELETE
- **Headers** — Authentication, Content-Type, etc.
- **Request Schema** — Input data structure
- **Response Schema** — Output data structure
- **Error Responses** — Standard error codes and messages

Example: User Resource Contract

Endpoint

```
GET /api/v1/users/{id}
```

Description: Retrieve user by ID

Request

Headers:

- Authorization: Bearer <token>
- Accept: application/json

Path Parameters:

Name	Type	Required	Description
id	string	Yes	User ID

Response (200 OK)

Content-Type: application/json

```
{
  "id": "123",
  "name": "John Doe",
  "email": "john@example.com",
  "status": "active",
  "createdAt": "2024-06-01T10:00:00Z"
}
```

Error Responses

Status Code	Description	Example
400	Bad Request	{ "error": "Invalid input" }
401	Unaathorized	{ "error": "Unauthorized" }
404	Not Found	{ "error": "User not found" }
500	Internal Server Error	{ "error": "Server error" }

Best Practices

- **Versioning** — Use URI versioning (e.g., /api/v1/)
- **Consistent Naming** — Use clear and consistent naming conventions
- **Filtering & Pagination** — Support query params for filtering, sorting, pagination
- **Security** — Use authentication and authorization
- **Documentation** — Keep API docs up to date (OpenAPI / Swagger)

VERSIONING API CONTRACTS

Evolve APIs without breaking consumers — choose a strategy, apply a documented compatibility policy, and keep a clear lifecycle.

STRATEGY	EXAMPLE	TRADE-OFF
`URI Path`	/api/v1/users	Easy to understand, cache-friendly
`Query Parameter`	/api/users?version=1	Easy to add, but may complicate routing, caching, documentation, and client consistency.
`Custom Header`	X-API-Version: 1	Less visible and harder to discover; requires explicit client headers; can complicate caching and observability.
`Media Type`	Accept: application/vnd.acme.user+json;v=1	Flexible, more complex to implement

VERSIONING API CONTRACTS — SEMANTIC VERSIONING

Apply a documented compatibility policy and reserve major versions for incompatible changes.

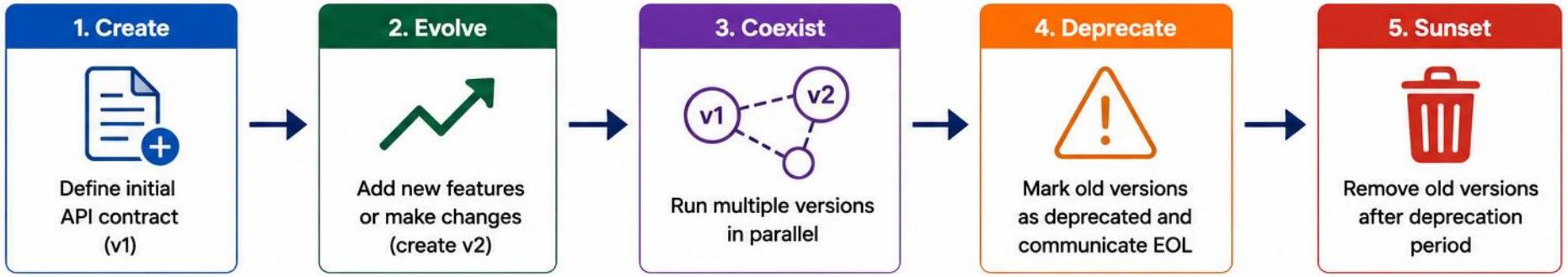
SEMANTIC VERSIONING

- Use a documented compatibility policy. Public HTTP APIs commonly expose only major versions, while internal releases may still follow semantic versioning.
- Deprecate, don't delete immediately — set clear sunset dates and provide migration guides.
- Do not create a new version for every additive change. Reserve new major versions for incompatible contract changes. Additive optional fields, new endpoints, new enum values, changed validation constraints, and changed error behavior are usually not version-triggering — though some "additive-looking" changes can still break strict clients.

KEY TAKEAWAY Versioning is about managing change, not just changing the URL — communicate clearly and evolve safely.

Versioning API Contracts

API Version Lifecycle



Versioning Strategies

URI Versioning

`/api/v1/users`

Example:

```
GET /api/v1/users
```

- Clear and explicit
- Most common

Header Versioning

`X-API-Version: 1.0`

Example:

```
X-API-Version: 1.0
```

- Keeps URI clean
- Requires custom header

Query Parameter Versioning

`?version=1`

Example:

```
GET /api/users?version=1
```

- Easy to implement
- Visible in URL

Media Type Versioning

`Accept: application/vnd.myapi.v1+json`

Example:

```
Accept: application/vnd.myapi.v1+json
```

- Follows content negotiation
- Best for complex APIs

Versioning Comparison

Strategy	Example	Pros	Cons	Best Use Case
URI Versioning	/api/v1/users	Simple, clear, easy to cache	URI changes, may clutter paths	Public APIs, most common choice
Header Versioning	X-API-Version: 1.0	Clean URI, flexible	Less visible, harder to test	Internal APIs, advanced clients
Query Parameter	?version=1	Easy to implement, visible	Affects caching, less RESTful	Simple APIs, quick adoption
Media Type Versioning	application/ vnd.myapi.v1+json	RESTful, uses content negotiation	Complex to implement, not widely used	Enterprise APIs, content-aware clients

Best Practices

- Use semantic versioning (v1, v2, v3...)
- Follow deprecation policy and timelines
- Communicate changes clearly
- Ensure backward compatibility when possible
- Document each version separately
- Monitor usage and plan version sunset

ENTERPRISE VALIDATION BEST PRACTICES (1 OF 2)

Validation is the first line of defense against bad data, security issues, and system failures.

Validate Early & Consistently

- At the API boundary, before business logic runs.

Defense in Depth

- Gateway (size, protocol, abuse controls); API boundary (structural DTO validation); Service/domain (business rules); Database (integrity constraints).

Comprehensive Constraints

- Null/empty, format, range, custom rules.

ENTERPRISE VALIDATION BEST PRACTICES (2 OF 2)

Validation is the first line of defense against bad data, security issues, and system failures.

Secure Your APIs

- Validate formats; encode or parameterize at the point of use (SQL, HTML, URLs, file paths).

Support i18n

- Externalize messages for multiple languages.

Test Your Validations

- Unit, negative, contract, and edge-case tests.

KEY TAKEAWAY Great validation leads to better data, happier consumers, fewer defects, and more reliable enterprise systems.

LAB OVERVIEW – DTOS AND VALIDATION

Design Customer DTOs and apply Jakarta Bean Validation programmatically for the Northstar CRM contract boundary.

WHAT YOU WILL LEARN

- Design CustomerRequestDTO and CustomerResponseDTO for create/read flows.
- Use common validation annotations (@NotNull, @NotBlank, @Email, @Size).
- Trigger validation programmatically and map entity to DTO without leaking fields.

PREREQUISITES

- JDK 21 and Maven 3.9+ (no Spring Boot in this lab).
- jakarta.validation-api, Hibernate Validator, and a compatible Jakarta Expression Language implementation if required by the selected configuration (e.g. org.glassfish:jakarta.el).
- Working CRM project (Customer, CustomerStatus) from Labs 9-12.

LAB OVERVIEW – HOW IT WORKS & VALIDATION BOOTSTRAP

The four-step validation flow, and the programmatic bootstrap code you'll use in Lab 14.

HOW IT WORKS

PROGRAMMATIC VALIDATION BOOTSTRAP

```
ValidatorFactory factory = Validation.buildDefaultValidatorFactory();
Validator validator = factory.getValidator();
Set<ConstraintViolation<CustomerRequestDto>> violations = validator.validate(request);
try (ValidatorFactory factory = Validation.buildDefaultValidatorFactory()) { Validator validator =
factory.getValidator(); }
```

KEY TAKEAWAY Always validate before mapping to the entity, and keep the correlation ID visible on failures.

TRY IT YOURSELF — EXERCISE 5: VALIDATOR TODOS (STARTER)

Reinforces: the programmatic validation bootstrap you just saw — fill-in-the-blank, not from scratch.

STEP	WHAT TO DO
Goal	FILL-IN-THE-BLANK STARTER — complete the ValidatorFactory checklist (no Spring @Valid)
File	notes/lab14-validator-todos.md (in examples/module-14-exercises/)
Fill in	Bootstrap blanks: buildDefaultValidatorFactory(), getValidator(), violation counts
Invalid cases	Add: blank fullName; unknown status; null customerId on activate
Self-check	Confirm Spring @Valid blank is answered "no" for this pre-lab
Reference	exercises/exercise-05-fill-validatorfactory-todos.md

```
ValidatorFactory factory = _____;   Validator validator = _____;
Invalid blank name -> expect _____ violations
Invalid status TYPO -> expect _____
Valid Amina ACTIVE sketch -> expect _____ violations
Spring @Valid in this pre-lab? _____
```

TRY IT NOW Complete Exercise 5 (STARTER) — see exercises/exercise-05-fill-validatorfactory-todos.md.

TRY IT YOURSELF — EXERCISE 6: LAB 14 PREP CHECKLIST

Reinforces: everything from Exercises 1-5 — confirm you're ready for Lab 14.

STEP	WHAT TO DO
Goal	Confirm DTO/validation prep is complete without doing Lab 14 itself
File	notes/lab14-prep-checklist.md (in examples/module-14-exercises/)
Artifacts	Entity/DTO note, paper DTO, mapper rules, and validator TODOs all exist
Boundary	DTOs before deep service rules; no Spring @Valid live in this prep
Fixtures	Recall the two fixtures: Amina ACTIVE and Ravi PROSPECT
Reference	exercises/exercise-06-lab14-prep-checklist.md

```
[ ] notes/lab14-entity-vs-dto.md      // Exercise 1
[ ] notes/lab14-paper-dto.md         // Exercise 2
[ ] notes/lab14-mapper-rules.md       // Exercise 3
[ ] notes/lab14-validator-todos.md    // Exercise 4
[ ] notes/lab14-invalid-cases.md      // Exercise 4 -- pass requires this one
```

TRY IT NOW Complete Exercise 6 before continuing — see exercises/exercise-06-lab14-prep-checklist.md.

LAB TASKS AND DELIVERABLES (1 OF 2)

Design Customer DTOs, validate programmatically, map safely, and prove rejection of invalid payloads.

#	TASK	DETAILS	FORMAT
1	Project Setup	Copy lab12-crm to lab14-crm; add Jakarta Validation deps.	n/a
2	Design DTOs	CustomerRequestDTO (validated) + CustomerResponseDTO.	.java
3	Implement CustomerMapper	Map Customer to/from DTOs; never leak entity fields.	.java

LAB TASKS AND DELIVERABLES (2 OF 2)

Validation, testing, and documentation — tasks 4-6, plus the success criteria for Lab 14.

#	TASK	DETAILS	FORMAT
4	Validate in CustomerApiFacade	Validator.validate() before service.createCustomer().	.java
5	Prove Rejection with Tests	JUnit tests: invalid email, blank name, valid Amina Khan.	.java
6	Document the Contract	README validation table; correlation ID lab-request-001.	.md

SUCCESS CRITERIA

- Valid CustomerRequestDTO maps and saves via the facade; invalid payloads never reach CustomerService; tests are green.
- Mapping must not allow the client to set customerId, createdAt, internal status flags, audit fields, or other server-controlled data.
- Tests cover: a null request field, a maximum length boundary, nested object validation with @Valid, mapping that doesn't leak internal fields, an invalid DTO that never reaches the service, and a stable error code with correlation ID.

KEY TAKEAWAY Well-designed DTOs and effective validation ensure data quality and a trustworthy API contract boundary.

DTO & API CONTRACT: DEFINITION OF DONE

Use this checklist to confirm a DTO and API contract are ready to ship.

1. Request and response models are separate where responsibilities differ.
2. Only consumer-required fields are exposed.
3. Server-controlled fields cannot be over-posted.
4. Validation rules are explicit and tested.
5. Structural and business validation are separated.
6. Mapping is intentional and asymmetric.
7. Errors use stable codes and safe messages.
8. Nullability and optionality are documented.
9. Versioning and compatibility impact are reviewed.
10. Examples and schema documentation match implementation.
11. Correlation IDs support troubleshooting.
12. Sensitive/internal fields never leave the boundary.

KEY TAKEAWAY **Run every DTO and API contract through this checklist before it ships.**

KNOWLEDGE CHECK (1 OF 2)

Test your understanding of DTOs and Jakarta Validation.

1. What's the primary purpose of a DTO?

A. To represent database tables
B. To carry data between client and server
C. To handle business logic
D. To manage transactions

2. Which annotation ensures a field is not null?

A. @NotNull
B. @NotEmpty
C. @NotBlank
D. @Size

3. Which annotation ensures a String isn't blank?

A. @NotEmpty
B. @NotNull
C. @NotBlank
D. @Required

4. Which annotation validates email format?

A. @Email
B. @Pattern
C. @ValidEmail
D. @Format

KEY TAKEAWAY DTOs carry data across boundaries; @NotNull, @NotBlank, and @Email are core Jakarta Validation annotations.

KNOWLEDGE CHECK (2 OF 2)

Four more questions, plus the full answer key.

5. How is validation triggered in a standalone app?

A. Add @Valid to the controller method

B. Obtain a Validator from ValidatorFactory and call validate()

C. Enable spring.validation.enabled

D. Annotate the class with @Validated

6. Does @Email make a field required?

A. Yes, it implies @NotNull

B. No — combine with @NotBlank or a presence constraint

C. Only when combined with @Size

D. Yes, but only for String fields

7. Which annotation validates a collection's size?

A. @Count

B. @Size

C. @Range

D. @Length

8. What's the typical status code for validation errors?

A. 200 OK

B. 400 Bad Request

C. 401 Unauthorized

D. 500 Internal Server Error

ANSWER KEY

- **1-B 2-A 3-C 4-A 5-B 6-B 7-B 8-B**

KEY TAKEAWAY Use the status your organization's API standard defines — 400 is common for validation, some APIs use 422; consistency matters more than one universal answer.

"Validate early, fail fast, build reliable APIs."

MODULE 14 COMPLETE

DTOs, Validation and API Contracts

You can now design DTOs, apply Jakarta Validation, and define clear, versioned API contracts with confidence.